

Insight Ladder — Day 3

Select an answer you cannot list

Priyanka | September 23, 2026 | Hard relative to this week | 120 minutes

Main problem

For every index pair $i < j$, form $|a[i] - a[j]|$. Return the k th smallest entry in this multiset of distances, counting equal distances separately. The rank k is one-based. Input: $2 \leq n \leq 200,000$, integer values from $-1,000,000,000$ to $1,000,000,000$, and $1 \leq k \leq n(n-1)/2$. Do not build the full pair-distance list. Target time: $O(n \log n + n \log(W+1))$, where $W = \max(a) - \min(a)$; $O(n)$ auxiliary storage is allowed. Use wide counts and differences in C.

Examples and checks: $[1,3,1]$, $k=1$ gives 0; $k=2$ gives 2; $k=3$ gives 2. $[4,4,4]$ gives 0 for every valid rank. $[0,4,9]$, $k=2$ gives 5. Test the smallest and largest ranks.

Optional variation

Reasoning task, not another coding quota: for $[1,1,3,3]$, list all six distances. Explain why a counting function can jump past k and why testing whether its count equals k is wrong. Give the output for $k=3$. Distinguish a rank among occurrences from a rank among distinct values.

Before reading hints: If you cannot cheaply list the answers, what cheaper question about a proposed answer would let you rule out many possibilities?

Your 120 minutes

- 0–10: closed-notes recall of one invariant or failed assumption from the previous session.
- 10–100: solve the main problem. Write a direct solution, identify repeated work, state a claim, try to break it, then implement. Attempt the optional variation only when the main solution is secure; it stays inside these 90 minutes.
- 100–120: stop coding, explain correctness and cost, review evidence, and log partial work. A full solution read during review does not turn an unfinished timed attempt into a solve.
- Use C11 OR Python 3 consistently. After 15 minutes without a useful new step, record the blocker and request one hint or continue. Record the time and level of every hint or appendix consultation.

Submit your evidence

Submit code, tests, the key claim and its justification, runtime/space, minutes spent, and hints used. Main status: independent solve / assisted solve / partial / unattempted. Insight: independent / after small hint / after method supplied. Track variation coding or reasoning separately; a counterexample is not another code solve. Count a main solve only when working code meets the contract and you can justify it. There is no completion quota.

Solution appendix

Open only after an attempt; record what you read and when. Read one hint at a time. The explanation below supplies the method and cannot count as independent discovery.

Progressive hints

1. Ignore selecting k for a moment. For a candidate distance d , can you count how many pairs have distance at most d ?
2. Sort and process each right endpoint. Maintain the earliest left endpoint whose distance from it is at most d . How many pairs end here?
3. The count is nondecreasing in d . Find the first integer d whose count is at least k , not the first with count exactly k .

Main solution reasoning

Sort a . Define $C(d)$ as the number of index pairs with distance $\leq d$. Start $l=0$. For each r , advance l while $a[r]-a[l] > d$, then add $r-l$. The valid left endpoints form a contiguous suffix before r ; earlier endpoints are too far away. As r increases, l never needs to move left. Thus C takes $O(n)$ time and $O(1)$ scan space. Binary-search integers $[0, W]$: $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) / 2$; if $C(\text{mid}) \geq k$ set $\text{hi} = \text{mid}$, else $\text{lo} = \text{mid} + 1$. Larger d includes every pair counted at smaller d , proving monotonicity. Initially W is feasible and 0 is a valid lower bound. Each update retains the first feasible integer and strictly shrinks the interval. When $\text{lo} = \text{hi}$, that integer is the k th distance. Sorting takes $O(n \log n)$, followed by $O(n \log(W+1))$ work; $W=0$ needs no search iterations.

Variation review

The six sorted distances for $[1, 1, 3, 3]$ are $[0, 0, 2, 2, 2, 2]$. $C(0)=2$ and $C(2)=6$. Rank 3 has answer 2 although $C(d)$ never equals 3. Repeated entries occupy separate ranks because distinct index pairs produce them. A first-feasible search handles the jump correctly. One complete, justified main solve is a strong result for this day.

Exit ticket and repair

If you cannot cheaply list the answers, what cheaper question about a proposed answer would let you rule out many possibilities?

If the explanation is still unclear: Build $C(d)$ by hand for $[1, 1, 3, 3]$ at $d=0, 1, 2$ before retrying the search.

Log: main attempted __; independent solve __; assisted solve __; partial __; variation status __; insight assistance __; minutes to first useful claim __; total minutes __.